

Performance and Scalability Strategies

How to identify bottlenecks, optimize performance, and scale prototypes effectively in platform-based systems.

KEY TOPICS

Bottlenecks

Optimization

Scalability

Load Balancing



Learning Objectives

By the end of this module, you will be able to diagnose issues, implement fixes, and verify improvements in your platform prototypes.



DIAGNOSIS

Identify common performance bottlenecks in platform-based systems.



OPTIMIZATION

Apply optimization and scalability techniques to improve prototype performance.



ANALYSIS

Document and analyze the results of performance improvement strategies.

Quick Recall

Let's review key concepts from previous modules before we dive in.

01 Why is it important to sanitize inputs in your prototype?

02 What is the main purpose of a prototype in platform development?

03 Which architecture allows scaling by adding more servers?

04 Which environment is used to deploy smart contracts?

NETFLIX

Scaling Failures Case Study

"Netflix once suffered major outages when too many people streamed at once. Why did the platform slow down?"

 Pair Activity •  10 Minutes



Analyze

Read the short Netflix scaling case study provided.



Hypothesize

Write down one possible reason for the slowdown.



Discuss & Share

Compare with a partner, select the most likely cause, and share.

Understanding Bottlenecks

Identifying the limiting factors in system performance is the first step toward optimization.



What is a Bottleneck?

A performance bottleneck is the **slowest part** of a system that limits the capacity or speed of the entire process.

 **Key Indicator:** One component hitting 100% capacity while others sit idle.



Database Bottleneck

Occurs when unoptimized queries scan too many rows or when indexes are missing, causing slow data retrieval.

Query Optimization



Code Inefficiency

Inefficient loops ($O(n^2)$), memory leaks, or repeated computations that consume excessive CPU resources.

Refactoring



API Bottleneck

Too many synchronous requests sent at once, hitting rate limits or causing network congestion.

Rate Limiting



Blockchain Gas Costs

Smart contract functions with complex logic or expensive storage operations leading to high transaction fees.

Gas Optimization

Scalability Strategies



When systems grow, performance shouldn't suffer. Choose the right strategy to handle increased load.



Vertical Scaling

Often called "Scaling Up." Involves adding more power (CPU, RAM, Storage) to your existing machine.

SIMPLER BUT LIMITED



Horizontal Scaling

Often called "Scaling Out." Involves adding more servers to your pool to distribute the workload.

COMPLEX BUT LIMITLESS



Load Balancing

The traffic cop. Efficiently distributes incoming network traffic across multiple servers.

TRAFFIC MANAGEMENT



⚠ The "Fail Whale" Era

Problem: Early Twitter was a monolithic Ruby on Rails app. When millions tweeted at once (e.g., World Cup), the single system crashed, showing the famous "Fail Whale."

✅ The Engineering Fix

MINI-CASE: TWITTER

Solution: They moved from a monolith to distributed **microservices** (Horizontal Scaling) and implemented sophisticated **load balancing** to handle spikes.

Session 1 Activity: Prototype Monitoring

🕒 35 Minutes

Gather evidence from your builds. Measure how fast or costly your prototype runs to identify bottlenecks.

AI Track Instructions

1. Open your prototype in Colab or VS Code.
2. Add timing code around one critical function:

```
import time
start = time.time()
# run your function here
end = time.time()
print("Time:", end - start)
```

3. Run at least 3 times with different inputs.

Blockchain Track Instructions

1. Open Remix IDE and load your contract.
2. Run a function (e.g., setMessage).
3. Check "Gas cost" in the console output.
4. Record costs for 3 different test cases.

Sample Performance Table

Test Case	Result	Identified Bottleneck
Function run	3.2 sec	Slow loop iteration
API call	2.5 sec	Network request delay
Contract function	50,000 gas	Expensive storage op

Grading Rubric (HPS = 50)

46-50 Points

3+ entries recorded, bottleneck clearly explained and analyzed.

40-45 Points

3 entries recorded with partial notes or limited analysis.

30-39 Points

Less than 3 entries recorded or explanations are unclear.

0-29 Points

Minimal attempt or missing data.



Session 2: Plan Your Fix

Before optimizing, let's recall key concepts and define your strategy for improvement.

Quick Recall

QUESTION 1

What is a bottleneck in a system?

QUESTION 2

Name one optimization technique to reduce load time.

QUESTION 3

What does "horizontal scaling" mean?

QUESTION 4

Why is it important to store API keys securely?

Main Task: Identify & Plan

1

Identify the Bottleneck

Review your monitoring table from Session 1. Choose one clear performance issue (e.g., slow API call or high gas cost).

2

Define Your Fix

Write one sentence describing exactly how you will solve it.

3

Get Approval

Show your written plan to your teacher before writing any code.

Optimization Techniques

Optimization means applying small, smart changes to make your prototype faster or more efficient. Here are common fixes used by engineers.

TYPE OF ISSUE	EXAMPLE FIX	WHY IT WORKS
 Slow Loops	Simplify / Refactor	 Reduces Computation Eliminates unnecessary repeated calculations per iteration.
 Slow Database Query	Add Indexing + Caching	 Speeds Retrieval Finds data instantly instead of scanning every record.
 Expensive Smart Contract	Combine Variables	 Lowers Gas Cost Reduces the number of write operations to the blockchain.
 High Latency	Asynchronous Requests	 Parallel Execution Lets multiple tasks run at once instead of waiting

Implementation & Results

Did your fix work? Measure the impact by comparing performance metrics before and after optimization.

Action Steps

- 1

Apply Fix
Re-run your code after adding caching, refactoring loops, or optimizing contracts.
- 2

Record Data
Measure the execution time or gas cost again. Log the "After" value.
- 3

Analyze
Calculate the difference. Write 2-3 sentences describing the improvement.
- 4

Share
Post your Before/After table to the class board.

Performance Comparison Example

 Student Example Data

TEST CASE	BEFORE	AFTER	IMPROVEMENT
 Chatbot Response	3.2 sec	1.9 sec	↓ Faster by 1.3 sec
 Contract Execution	50,000 gas	35,000 gas	🌱 Saved 15,000 gas

Grading Rubric (50 pts)

46-50

Fix applied correctly. Before/after results clear. Improvement is significant.

40-45

Fix applied. Results shown. Improvement is moderate or noted.

30-39

Fix incomplete or documentation is weak/unclear.

0-29

Minimal attempt. Missing data or no fix applied.



Key Takeaways

Summary of the essential concepts from this module on performance and scalability.



LIMITATIONS

Bottlenecks limit system performance.



DETECTION

Monitoring reveals slow points before failure.



OPTIMIZATION

Small fixes (caching, refactoring) can make big improvements.



GROWTH

Scalability ensures apps can grow with more users.



PROCESS

Performance optimization, like security, is a continuous process.

References & Further Reading

Explore these engineering blogs and documentation to deepen your understanding of real-world scalability strategies.



Netflix Tech Blog

"Scaling Lessons Learned" - Deep dives into chaos engineering and handling massive concurrent traffic spikes.

netflixtechblog.com →



Google Cloud Architecture

"Performance Best Practices" - Comprehensive guides on optimizing cloud infrastructure and database performance.

cloud.google.com/architecture →



AWS Documentation

"Simple Auto Scaling Guide" - Learn the fundamentals of setting up elastic infrastructure that grows with demand.

aws.amazon.com/autoscaling →



Twitter Engineering

"The Fail Whale Era" - A classic case study on migrating from a monolithic architecture to microservices.

blog.twitter.com/engineering →